CS110 / INDEPENDENCE ROAD

reportB / George Arbache

Issued: Nov 12, 2026, 3:53 PM Pacific

Record: FORM-9B9D32B9F38F9408E3D9D2FB

Personal public verification code: VIEW-09B729640A64D42C5F0A885B

Include your own public code with every assignment, even in a team folder. Never submit your private recovery key.

SHA-256: a864f738b6fa50425dc81d416785e3f1da6674e1c07df2ecaf62bd80f4edefd6

Verify the record number at courses.addp2p.com/development-road/independence/index.html?view=verify

An issued record preserves submitted information. It is not a grade or proof of comprehension.

What did you personally execute? Connect completed tasks to files/commits.

I [built [feature, e.g., the player movement, the bounty selection screen, the criminal chase behavior] for our Bounty Hunting game. I worked in [file names, e.g., player.js, bounty_list.py] and [committed/saved it as commit message or file version].

This was my task from our huddle: [Bounty Hunting Game]. It supports the goal of turning the app into a game where the player hunts criminals.

I ran the game and [what happened, e.g., the player can now select a bounty and the target appears on the map]. Evidence: [screenshot, short clip, or test output].

[e.g., The target sometimes doesn t spawn after accepting a bounty. Everything works, and I will continue to test the game.

I used Claude Ai to get the answer.

Record setup/import checks, project opens/runs, player configuration/result, and one failure fix retest. Separate observed results from predictions.

I imported the project in [Unity, e.g., Unity, Godot, VS Code]. Observed: [it loaded with no errors.

Project runs: I ran the game by [how, e.g., pressing Play, python main.py]. Observed: [what appeared on screen, e.g., the main menu loaded and the player spawned].

Player result: I set up the player with [e.g., movement speed, controls, health, a bounty to hunt]. Observed: [what happened, e.g., the player moved with WASD and could select a bounty].

Failure: [exactly what went wrong, e.g., the target didn t spawn after accepting a bounty, plus any error message]

Fix: [what you changed, in which file, e.g., fixed the spawn call in bounty_manager.js]

Retest: [what happened after, e.g., the target now appears and can be captured]

I used Claude Ai.

Personal GitHub URL/commit, team link if used, project/source paths, PDFs, boards/images, credits and reproduction instructions.

CS110 / INDEPENDENCE ROAD

a

Compare the result with the mission. What did you verify, what remains, and what should the team do next?

Mission vs. result: Our mission was to convert the Bounty Hunting App into a game where the player actually hunts criminals. Compared with that, we now have [what exists, e.g., a playable level where the player accepts a bounty and chases a target].

What I verified (observed): [Things you tested and saw work, e.g., player movement works, accepting a bounty spawns a target]. Evidence: [screenshot, commit, test output].

What remains: [Things not done or not tested, e.g., capturing the target and the reward payout aren t built yet, only one level exists].

Useful discovery: [Something you learned that helps the team, e.g., the old app s bounty data can be reused directly for spawning targets, or the chase logic needs to be separate from the UI or it breaks].

What the team should do next:

[Highest priority task, who should do it]

[Next task, including any dependency]

[How we ll check it works, e.g., a test run together before the next huddle]

I used Claude Ai.

Name

George Arbache

Action

B

Issued team form

FORM-816A8E94AA67ED10B8A60E5F

Whiteboard

The Beginning Shows the search engine which is Unity.

The Bounty is a video game, where we catch bad guys who are wanted which is by clicking play, which leads to the main menu, to the bounty.

The Failure is the target wasn't appearing up on the screen, plus an error message.

The solution to the fix is we changed it to bounty_mamager.js

Result is the target showed up for the bounty to be capture.

Member time receipts

0

Name

nicholas kuhnast

verificationCode

VIEW-C7B7557832002B7EF5EC958C

A

Time code

TIME-A38804FF9D661924E56FA2B1

Name

nicholas kuhnast

Action

A

Server arrival

2026-11-10T20:58:37.522Z

Class starts

2026-11-10T20:45:00Z

Calculated late points

10

Late deduction points

10

Review status

Calculated awaiting instructor review

CS110 / INDEPENDENCE ROAD

recovery

Review status

Not requested

requestedAt

restoredPoints

0

approvedAt

B

Time code

TIME-E30F2B9CE0297D02520C60A4

Name

nicholas kuhnast

Action

B

Server arrival

2026-11-12T20:45:46.431Z

Class starts

2026-11-12T20:45:00Z

Calculated late points

0

Late deduction points

0

Review status

Calculated awaiting instructor review

recovery

Review status

Not requested

requestedAt

restoredPoints

0

approvedAt

1

Name

George Arbache

verificationCode

CS110 / INDEPENDENCE ROAD

VIEW-09B729640A64D42C5F0A885B

A

Time code

TIME-1899E24EC2247D32DA8BFFC7

Name

George Arbache

Action

A

Server arrival

2026-11-10T20:58:20.477Z

Class starts

2026-11-10T20:45:00Z

Calculated late points

10

Late deduction points

10

Review status

Calculated awaiting instructor review

recovery

Review status

Not requested

requestedAt

restoredPoints

0

approvedAt

B

Time code

TIME-9607CDD2B90AEF9E48065171

Name

George Arbache

Action

B

Server arrival

2026-11-12T21:48:18.001Z

Class starts

2026-11-12T20:45:00Z

Calculated late points

CS110 / INDEPENDENCE ROAD

60

Late deduction points

20

Review status

Calculated awaiting instructor review

recovery

Review status

Not requested

requestedAt

restoredPoints

0

approvedAt

2

Name

Christopher Guillen

verificationCode

VIEW-7567690B77A63DE3EFD55B26

A

Time code

TIME-19CD2B19DC20C853E0C08615

Name

Christopher Guillen

Action

A

Server arrival

2026-11-10T21:04:09.183Z

Class starts

2026-11-10T20:45:00Z

Calculated late points

15

Late deduction points

15

Review status

Calculated awaiting instructor review

recovery

Review status

Not requested

CS110 / INDEPENDENCE ROAD

requestedAt

restoredPoints

0

approvedAt

B

Time code

TIME-D75457E3E52A8D734587A258

Name

Christopher Guillen

Action

B

Server arrival

2026-11-12T21:32:04.046Z

Class starts

2026-11-12T20:45:00Z

Calculated late points

45

Late deduction points

20

Review status

Calculated awaiting instructor review

recovery

Review status

Not requested

requestedAt

restoredPoints

0

approvedAt

3

Name

Andy Jimenez

verificationCode

VIEW-768D7906597AD04E5494DAEB

A

Time code

CS110 / INDEPENDENCE ROAD

TIME-09AFF0BF44BC83DF0612A784

Name

Andy Jimenez

Action

A

Server arrival

2026-11-12T21:23:51.853Z

Class starts

2026-11-10T20:45:00Z

Calculated late points

2915

Late deduction points

20

Review status

Calculated awaiting instructor review

recovery

Review status

Not requested

requestedAt

restoredPoints

0

approvedAt

B

Time code

TIME-3900FA53A4848ED2C5FBFB10

Name

Andy Jimenez

Action

B

Server arrival

2026-11-12T21:25:18.288Z

Class starts

2026-11-12T20:45:00Z

Calculated late points

40

Late deduction points

20

CS110 / INDEPENDENCE ROAD

Review status

Calculated awaiting instructor review

recovery

Review status

Not requested

requestedAt

restoredPoints

0

approvedAt

4

Name

Leah Aluni

verificationCode

VIEW-015379456E7877924D39881E

A

Time code

TIME-551E796C1E514BEF8F479959

Name

Leah Aluni

Action

A

Server arrival

2026-11-12T21:24:14.934Z

Class starts

2026-11-10T20:45:00Z

Calculated late points

2915

Late deduction points

20

Review status

Calculated awaiting instructor review

recovery

Review status

Not requested

requestedAt

restoredPoints

CS110 / INDEPENDENCE ROAD

0

approvedAt

B

Time code

TIME-347E1E5B735E5BC9D24C2AB7

Name

Leah Aluni

Action

B

Server arrival

2026-11-12T21:04:22.791Z

Class starts

2026-11-12T20:45:00Z

Calculated late points

15

Late deduction points

15

Review status

Calculated awaiting instructor review

recovery

Review status

Not requested

requestedAt

restoredPoints

0

approvedAt

workNotes

My next task is to help develop our Bounty Hunting Game using Unity and C#. I plan to focus on creating the basic player movement system and developing the mechanics that will allow players to find, chase, and capture criminals. I will start by creating a player GameObject in Unity and using C# scripts to control movement, including walking, running, and changing directions. I also want to experiment with Unity's collision detection system so the player can interact with criminals when they get close enough.

After creating the movement system, I plan to work on a basic criminal capture mechanic. This could involve programming a criminal character that moves around the map and allowing the player to capture the criminal by pressing a specific key when within a certain distance. I will also need to test these features in Unity's Play Mode to make sure the controls work correctly and the capture system responds as expected.

My goal is to make progress on these features before our next class deadline. One possible blocker is that I am still

learning how to write C# scripts and connect them to GameObjects in Unity. I might also encounter problems with movement, collisions, or programming the criminal's behavior.

If I run into difficulties, I will review Unity tutorials, study C# programming examples, ask my teammates for assistance, and experiment with different solutions. I will use Unity's Console window to identify errors and debug my scripts. Once the features are working, I plan to save my progress, upload the updated project files to GitHub, and document what I completed, what problems I encountered, and what still needs improvement. My main priority is to help create a simple playable prototype that we can continue improving as a team.

I understand that our main goal is to turn the Bounty Hunting App into a playable video game using Unity and C#. In our game, players will be able to explore different locations, search for wanted criminals, chase them, and capture them to complete missions and earn rewards. We want to create an interactive experience where players can control their character, interact with the environment, and complete different bounty hunting objectives.

My responsibility is to help my team plan, design, and program the game, especially the player movement and criminal capture systems. I plan to learn how to use Unity's GameObjects, components, and C# scripts to create a character that can walk, run, and interact with other objects. I also want to help develop a system where criminals can move around the map and react when the player approaches them. For example, criminals could attempt to escape, requiring the player to chase them before making a capture.

Another important part of our project is creating the capture mechanics. We need to decide how close the player must be to a criminal and what action they must perform to capture them. We could use Unity's collision detection system or distance calculations to determine when a criminal is within capture range. After a successful capture, the game could display a notification, update the player's mission progress, and award points or money.

We will also use a project board to organize our responsibilities and track our development progress. The board will separate tasks into categories such as To Do, In Progress, Testing, and Completed. This will help our team understand which features still need to be developed and which ones are ready for testing. We also plan to use GitHub to save our Unity project files, track code changes, and collaborate with teammates.

There are still several important decisions we need to make. We need to determine whether the game will be 2D or 3D, how the player controls will work, how criminals will behave, and how missions will be completed. We also need to decide which teammates will be responsible for programming, designing the environment, creating the user interface, and testing the game.

My next priority is to help create a simple playable prototype in Unity. The prototype should allow a player to move around a basic environment, locate a criminal, and perform a capture action. Once these basic mechanics are working, our team can begin adding more advanced features such as criminal AI, multiple missions, different locations, and a reward system. I understand that developing the game will require teamwork, problem-solving, testing, and learning more about Unity and C# programming.

I created a detailed project board for our Bounty Hunting Game to help organize our goals, development tasks, team responsibilities, and next steps. Our main objective is to transform the original Bounty Hunting App into an interactive video game using Unity and C#. In the game, players will be able to explore different environments, search for wanted criminals, chase them, and capture them to complete missions and earn rewards.

The project board organizes our development process into four main categories: To Do, In Progress, Testing, and Completed. I included important tasks such as creating player movement controls, programming criminal behavior, developing capture mechanics, designing the game map, and testing gameplay features. This organization will help our team understand what needs to be completed, what we are currently working on, and what requires additional improvements.

I also identified the programming tools and systems we may need to develop the game. We plan to use Unity as our

game engine and C# to program the gameplay mechanics. Unity's GameObjects and components will allow us to create characters, environments, and interactive objects. C# scripts will help us control player movement, criminal behavior, and the capture system. We may also use Unity's physics and collision detection features to determine when players interact with criminals.

Another important part of my planning was identifying possible programming challenges. Some difficulties we might encounter include writing C# scripts correctly, making characters move smoothly, programming criminals to escape from the player, and ensuring the capture system works properly. To overcome these challenges, I plan to review Unity tutorials, experiment with different programming solutions, use the Unity Console to identify errors, and communicate with my teammates when I need assistance.

Our next step is to begin developing a basic playable prototype in Unity. We should start by creating a simple environment and a player character that can move around using keyboard controls. After that, we can add a criminal character and develop a basic capture system that allows the player to arrest the criminal when they are within a certain distance. Once these features are implemented, we will test them in Unity's Play Mode, identify any bugs, and make improvements.

I also plan to use GitHub to save our project files, track changes, and document our development progress. Overall, creating the project board helped me better understand the steps required to develop our game and how we can divide the work among team members. It gives us a clear development plan and will help us stay organized as we continue building and improving our Bounty Hunting Game.

I created a project board for our Bounty Hunting Game to organize the different tasks and features we need to develop using Unity and C#. The board includes important parts of the game, such as player movement, criminal capture mechanics, map design, criminal behavior, and gameplay testing. I organized these tasks into categories such as To Do, In Progress, Testing, and Completed so our team can keep track of our progress and understand what still needs to be done.

Although I have completed the initial planning and organization of our project, I have not tested the actual game in Unity yet. My next step is to make sure our Unity project opens correctly, that the game scene loads without errors, and that the basic gameplay features function as intended. I plan to use Unity's Play Mode to test the game and the Console window to check for programming errors or warnings.

One of the first features I want to test is player movement. We plan to use C# scripts to allow the player to move around the environment using keyboard controls. I will need to check whether the character moves in the correct directions, responds properly to player input, and interacts with the environment without unexpected problems. If we use Unity's Rigidbody or Character Controller components, I will also need to make sure movement and collision detection work correctly.

Another important feature to develop and test is the criminal capture system. Our goal is to allow players to locate criminals, chase them, and capture them when they are within a certain distance. We could use C# scripts and Unity's collision detection or distance calculations to determine when a criminal is close enough to capture. I will need to test whether the capture action activates correctly, whether the criminal is registered as captured, and whether the game updates the mission progress.

I also plan to test the game environment to make sure the player can move around the map without getting stuck or passing through objects. If we add criminal AI, we will need to check whether criminals move correctly and react when the player approaches them. These tests will help us identify problems before adding more advanced gameplay features.

If something does not work correctly, I will try to identify the cause by reviewing the C# scripts, checking the Unity Inspector settings, and reading error messages in the Console. After making changes, I will run the game again to see whether the problem has been fixed. I will document what worked, what failed, what changes were made, and

whether the retest was successful.

My next priority is to help build a simple playable prototype where the player can move around a basic environment, find a criminal, and perform a capture action. Once we have tested these basic features successfully, our team can begin adding more advanced mechanics, including criminal AI, additional missions, rewards, and improved map design. I also plan to save our progress to GitHub so we can track changes and collaborate more effectively.

Overall, the project board has helped me understand the development process and prepare for the testing stage. While we have completed some of the planning work, we still need to implement and verify the actual gameplay systems. Our goal is to make sure the basic mechanics work correctly before expanding the game further.

I created a project board for our Bounty Hunting Game and compared it with our original goal of developing an interactive video game where players can search for, chase, and capture wanted criminals. Our team plans to use Unity as the main game engine and C# as the programming language to create the gameplay mechanics. By reviewing our project board, I was able to confirm that our development plan includes the important features needed to make the game playable, such as player movement, criminal capture mechanics, map design, criminal behavior, and gameplay testing.

One of the main things I verified was that our project plan focuses on creating the basic gameplay features before adding more advanced systems. For example, we need to create a player character in Unity that can move around the environment using keyboard controls. We also need to develop a criminal character that the player can locate and capture. These features will require us to use Unity's GameObjects, components, and C# scripts to control how characters move and interact with each other.

I also reviewed our plans for the criminal capture system. We want players to be able to approach criminals and perform a capture action when they are within a certain distance. One possible solution is to use Unity's collision detection system or C# distance

Task list

0

id

e9fcadfb-346c-4e46-81be-35b6fba97260

task

make assest

Task owner reference

nicholas kuhnast

Deadline

2026-11-12T20:30

Completed

false

Execution / result

1

id

ab8b7007-c585-474b-b404-450d07460f53

task

do the coding

CS110 / INDEPENDENCE ROAD

Task owner reference

nicholas kuhnast

Deadline

2026-11-12T20:30

Completed

false

Execution / result

2

id

5019ca2c-a540-4466-82bc-74fac46c30e6

task

Organize information

Task owner reference

Andy Jimenez

Deadline

2026-11-12T20:30

Completed

false

Execution / result

reviewedPlan

mission

We are going to convert the Bounty Hunting App into a game where the player will actually hunt for the criminals.

framework

We chose P2P.

pipeline

Unnamed stage

Unnamed stage

Unnamed stage

Unnamed stage

Unnamed stage

Unnamed stage

Placeholder owner Bounty Sketch

Placeholder owner Connector

We put everything in order and used Claude Ai to give us the answers

tech

Engine: Unity. It s free, lightweight, and has good beginner tutorials, so everyone can install it and start quickly.

CS110 / INDEPENDENCE ROAD

Language: Unity C#. It s built into Godot and easy to read, so teammates can understand each other s code.

Packages/add-ons: Godot s built-in 2D/3D tools (sprites, collision, UI nodes) so we don t depend on extra downloads that might break.

Assets: Free, license-friendly art and sound (for example from Kenney.nl or OpenGameArt), plus our own wanted-poster designs made in Canva or Figma. We chose free assets so we don t have copyright problems.

Who checks them:

[Leah Aluni] checks that the engine and code run on every team member s computer.
[Nicolas Kuhnast] checks that every asset has a license that lets us use it.
[George Arbache] tests the finished build after each change.

Why we chose these: They re free, beginner-friendly, and work well for a small team with limited time. Open question: Do we need any extra add-ons for saving scores, or is the built-in system enough? I used Claude to get the answer

Time code

Start / Update

Callback | What happens | Verified

Awake() | Sets up references (player, bounty list) before the game starts | [x]

Start() | Loads the first bounty and sets the score to 0 | [x]

Update() | Every frame: reads player input, moves the player, checks if the bounty is caught | [x]

OnCollisionEnter2D() | Runs when the player touches a bounty target, then adds the reward to the score | [x]

Classes & components

Class | Responsibility | GameObject

PlayerController | Reads input and moves the player; checks when the player touches a bounty | Player

BountyTarget | Stores each target s name, difficulty, and reward value | Bounty (prefab)

GameManager | Tracks the score and which bounty is active; decides when the game ends | GameManager (empty object)

UIManager | Updates the score text and shows the win or game over screen | Canvas

Packages / using

Package / namespace | Why included | Tested

using UnityEngine; | Gives access to the core engine tools our scripts need, like MonoBehaviour, Transform, Input, and Vector2 | [x]

using UnityEngine.UI; | Lets us update the score text and buttons on the screen from code | [x]

using System.Collections.Generic; | Lets us use List<> to store the bounty list and loop through it | [x]

using UnityEngine.SceneManagement; | Lets us reload the scene when the player restarts after winning or losing | [x]

communication

CS110 / INDEPENDENCE ROAD

Team check-ins

Description / contact / owner | When Pacific | Agreed

George Arbache | 2026-11-10T15:48 | [x]

reuse

PlayerController script (our own code). Source: written by our team. Credit: none required, but we ll keep our names in the code comments.

GameManager and score system (our own code). Source: written by our team. Credit: none required. It can be reused for any game that tracks points or objectives.

Bounty data structure (BountyTarget class). Source: our team. Credit: none required. We can swap in new characters, rewards, and difficulty levels for a new story.

Free art and sound assets (for example from Kenney.nl or OpenGameArt). Source: the website and creator s name, saved in a credits file. Credit: we ll check each license, since some (CC0) need no credit and others (CC-BY) require us to name the creator.

Our wanted-poster and menu designs (Canva or Figma). Source: our team. Credit: if we used Canva templates or stock elements, we ll check Canva s license terms before reusing them. I used Claude for my answer.

boardNotes

What the huddle board is: A shared space where our team writes down what we discussed, what we agreed on, and what we still need to figure out, so nobody has to rely on memory and everyone can see the same plan.

Decisions we made:

We will build the game in [Godot/Unity] using [GDScript/C#], because it s free and beginner-friendly.

We will keep the game small: at least 3 bounties, a score system, and a win or game over screen.

We will use free, license-friendly assets and keep a credits file listing each source.

Each person owns one part of the pipeline: [Name 1] inputs, [Name 2] building, [Name 3] design, [Name 4] testing.

Disagreements:

Some of us wanted more bounties and extra levels, while others wanted to finish a smaller game that works. We agreed to finish 3 bounties first and add more only if time is left.

We discussed whether to draw our own art or use free assets. We chose free assets for the characters and made our own wanted posters.

Questions that remain open:

Do we need an extra add-on to save high scores, or is the built-in system enough?

Who will double-check that every asset s license allows us to reuse it?

How many classmates can test the game before the deadline? I used Claude to get the answers

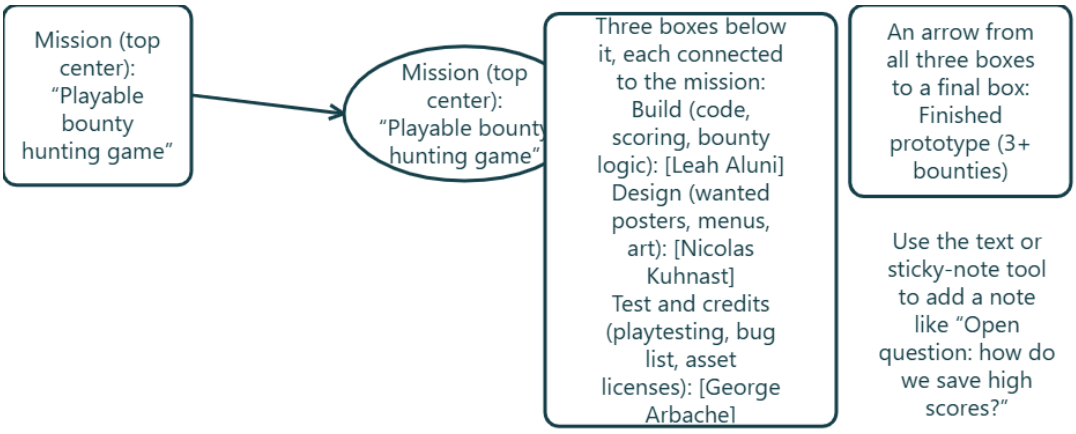
Whiteboard

Our mission is to build a playable bounty hunting game with at least three bounties, a score system, and a win or game over screen. In our huddle we compared our ideas for the mission and agreed to keep the game small so we

can finish it.

The diagram shows the mission at the top, connected to three jobs. [Name 1] builds the game and scoring. [Name 2] designs the posters, menus, and art. [Name 3] tests the game, keeps the bug list, and checks that every asset s license lets us use it. All three jobs lead to the finished prototype at the bottom.

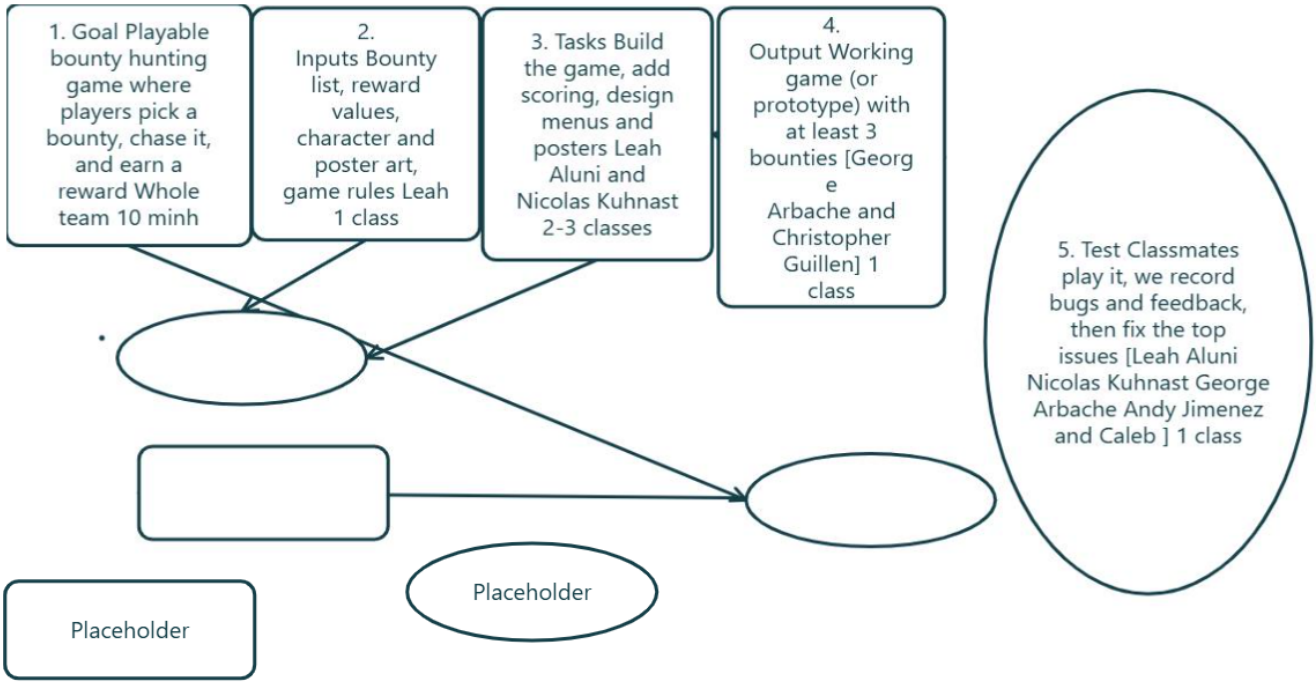
We decided that each person owns one job so nothing is missed, and that we will check in again before the deadline. One disagreement was whether to add more bounties or finish a smaller game first, and we chose to finish three bounties first. One open question is how we will save high scores. we used claude to get the answers



Whiteboard

We put everything in order and used Claude Ai to give us the answers

CS110 / INDEPENDENCE ROAD



codeTables

0

id

f755df56-99d0-432a-9e1f-74ecc1902297

title

Start / Update

columns

0

label

Callback

type

text

1

label

What happens

type

text

2

label

Verified

type

checkbox

rows

CS110 / INDEPENDENCE ROAD

0

0

Awake()

1

Sets up references (player, bounty list) before the game starts

2

true

1

0

Start()

1

Loads the first bounty and sets the score to 0

2

true

2

0

Update()

1

Every frame: reads player input, moves the player, checks if the bounty is caught

2

true

3

0

OnCollisionEnter2D()

1

Runs when the player touches a bounty target, then adds the reward to the score

2

true

1

id

c0b5405d-1eba-4c5e-8cd3-c3785ef3b8e8

title

Classes & components

columns

0

label

Class

type

text

CS110 / INDEPENDENCE ROAD

1

label

Responsibility

type

text

2

label

GameObject

type

text

rows

0

0

PlayerController

1

Reads input and moves the player; checks when the player touches a bounty

2

Player

1

0

BountyTarget

1

Stores each target s name, difficulty, and reward value

2

Bounty (prefab)

2

0

GameManager

1

Tracks the score and which bounty is active; decides when the game ends

2

GameManager (empty object)

3

0

UIManager

1

Updates the score text and shows the win or game over screen

2

Canvas

CS110 / INDEPENDENCE ROAD

2

id

964a5672-74ca-40a8-8427-ccc9b1295e38

title

Packages / using

columns

0

label

Package / namespace

type

text

1

label

Why included

type

text

2

label

Tested

type

checkbox

rows

0

0

using UnityEngine;

1

Gives access to the core engine tools our scripts need, like MonoBehaviour, Transform, Input, and Vector2

2

true

1

0

using UnityEngine.UI;

1

Lets us update the score text and buttons on the screen from code

2

true

2

0

using System.Collections.Generic;

CS110 / INDEPENDENCE ROAD

1

Lets us use List<> to store the bounty list and loop through it

2

true

3

0

using UnityEngine.SceneManagement;

1

Lets us reload the scene when the player restarts after winning or losing

2

true

communicationTables

0

id

94fbcf11-58d1-405b-b8c3-f121cee8b18c

title

Team check-ins

columns

0

label

Description / contact / owner

type

text

1

label

When Pacific

type

datetime-local

2

label

Agreed

type

checkbox

rows

0

0

George Arbache

1

2026-11-10T15:48

CS110 / INDEPENDENCE ROAD

2

true

Preparation record

PREP-FEDF5DDE03056AEE0AC5A35B

Contribution record

FORM-895E44D479923FA200AA8DE3

dayBReview

goal

true

owners

true

blockers

true

notes

What changed: We confirmed the goal is to turn the Bounty Hunting App into a game where the player actively hunts criminals, and that we re aiming to have it done by Video 6. Our plan changed from just adding features to the app to building real gameplay (choosing a bounty, tracking the target, capturing them, getting a reward) I used Claude Ai to get my answers.

[George Arbache] still needs to confirm [Bounty Hunting Game] by [Thursday November 12th 2026], and it depends on [Leah Aluni].

Possible blocker: [e.g., unsure how to build the chase mechanic / art assets aren t ready / Caleb Jackson].

We ll check results by [e.g., playing a test build together, or sharing a screenshot or short clip before the next meeting].

recap

What I understand: Our team already has a Bounty Hunting App. We re converting it into a game where the player actively hunts the criminals, instead of just browsing or tracking bounties. That means adding gameplay: the player picks a bounty, searches for or chases the target, and completes the capture to earn a reward.

My job: Idea pitcher/assistant, which is when I help Christopher Guillen with ideas up on the board and assisting him with writing ideas on the board

What I need to clarify:

What parts of the existing app do we keep (bounty list, criminal profiles, rewards), and what do we replace with gameplay?

What type of game is it (2D or 3D, top-down or side view)?

What counts as finished for our first version, such as one playable mission?

Who is responsible for each part, so we don t overlap?